

THE STOCHASTIC F1 RACE STRATEGIST

Author: Jerome Maunga

Degree: BSc Mathematics & Computer Science

Date: August 2026

1. Executive Summary

This project involves building a high-fidelity, stochastic race simulation engine tailored for Formula 1 strategy analysis. Unlike static calculators, this system leverages Monte Carlo methods to model tire degradation, fuel burn, and random external factors (Safety Cars) to output probabilistic win predictions. The final deliverable is an interactive web dashboard where users can compare pit-stop strategies and visualize win-probability distributions, effectively acting as a digital race engineer.

2. Problem Statement

In modern motorsports, particularly F1 and sim-racing, the difference between winning and losing is often determined by strategic calls regarding pit stops. Currently, amateur analysts and sim-racers rely on static spreadsheets or gut feeling to determine if a 1-stop or 2-stop strategy is faster.

The core problem is threefold:

- Static Models Fail:** Traditional calculators use fixed lap times and ignore the stochastic nature of racing (driver inconsistency, random Safety Car deployments).
- Tire Physics are Simplified:** Most models treat tire degradation as a linear loss of time, whereas real tires exhibit exponential "cliff" drop-offs and non-linear grip curves.
- Lack of Probabilistic Output:** A single "estimated race time" is statistically useless without a confidence interval or probability of success against rival strategies.

This project solves these issues by creating a Monte Carlo engine that runs tens of thousands of race iterations, providing a statistical distribution of outcomes rather than a single deterministic number.

3. Necessary Mathematical Units (Academic Syllabus)

To execute this project successfully, a deep understanding of the following university-level mathematics modules is mandatory:

Mathematics Unit	Specific Topics Required	Application in Project
------------------	--------------------------	------------------------

Unit		
Probability Theory	Random Variables (Uniform, Gaussian, Poisson); Expectation; Variance; Law of Large Numbers.	Modeling Safety Car rates (Poisson); simulating driver inconsistency (Gaussian); justifying convergence of Monte Carlo results (LLN).
Statistical Inference	Confidence Intervals; Standard Error; Central Limit Theorem; Hypothesis Testing.	Calculating 95% CI for finishing times; determining if win-rates are statistically significant.
Numerical Analysis	Floating-point precision; Error propagation; Euler's Method / RK4 Integration.	Accumulating lap times without floating-point drift; solving ODE for Tire Temperature vs. Grip.
Differential Equations (ODEs)	First-Order ODEs; Initial Value Problems; Rate of change modeling.	Modeling tire degradation as a function of heat cycles ($dGrip/dt = -k * Grip$).
Optimization Theory	Objective functions; Local vs Global Minima; Search Spaces.	(Advanced) Implementing a Genetic Algorithm to find the optimal pit-stop window automatically.

4. Necessary Computer Science Units (Academic Syllabus)

The coding architecture requires strict adherence to software engineering principles taught in the following CS modules:

Computer Science Unit	Specific Topics Required	Application in Project
Object-Oriented Programming (OOP)	Inheritance, Polymorphism, Encapsulation; Strategy Design Pattern.	Creating base classes with children overriding degradation methods; implementing Strategy Pattern for swapping pit-stop plans.
Parallel Computing	Amdahl's Law; Python GIL; Multiprocessing vs. Multithreading.	Using multiprocessing.Pool to spawn cores, reducing runtime from 10 minutes to ~90 seconds.
Data Structures & Algorithms	Time Complexity (Big-O); Hashmaps; Vectorized Operations (NumPy).	NumPy vectorization for lap-times; optimizing sorting for leaderboards across 100k simulations.
Discrete Event Simulation (DES)	Finite State Machines (FSM); Event Queues.	Building a State Machine for the car (RACING → PIT_IN → PIT_STOP → PIT_OUT) and handling SC deployment.
Database Systems	Relational Schemas; Time-Series Data; Indexing (SQLite).	Storing millions of lap-time data points; querying specific runs to replay race order dynamically.

5. System Architecture & Technical Stack

The stack below is optimized for a **solo developer on a 4-6 week timeline**: every choice favors development speed and low operational overhead over theoretical scalability. Where the original candidate list offered two options, the table states which one to build with and why.

Layer	Chosen Tool	Rationale
-------	-------------	-----------

Language	Python 3.10+	Only realistic choice — the NumPy/SciPy ecosystem is unmatched for this workload and maps directly onto OS/DSA coursework.
Core Math/Science	NumPy, SciPy	NumPy vectorization is the single biggest performance lever in the project. SciPy supplies <code>gaussian_kde</code> for the PDF overlay and <code>odeint</code> if the tire-grip ODE is implemented.
Data Handling	Pandas	Used to shape simulation output before it hits SQLite or Plotly — a convenience layer, not core logic.
Parallelization	<code>multiprocessing.Pool</code> (not Ray)	Ray adds a distributed scheduler you don't need for a single-machine, 8-core speedup. <code>Stdlib multiprocessing</code> gets the same ~90-second runtime with zero extra dependencies. Only revisit Ray if the project later needs multi-machine scaling.
Frontend/Dashboard	Streamlit (not Dash)	Streamlit trades fine-grained callback control for raw development speed — <code>st.slider()</code> vs. Dash's callback-graph boilerplate. Correct trade-off for a solo 4-week sprint with one user.
Charts	Plotly (in Streamlit); Matplotlib for scratch work	Plotly gives interactive hover/zoom for the dashboard's PDF histograms and lap-traces. Matplotlib stays as the throwaway tool for Sprint 1's quick validation plots.
Database	SQLite	Zero-config, file-based, comfortably fast for millions of rows on one machine, and directly exercises the Database Systems module.
Genetic Algorithm (Sprint 4)	Hand-rolled (not DEAP)	Building selection/crossover/mutation manually makes the Optimization Theory syllabus link concrete instead of hiding it behind a library — worth the extra day for a portfolio piece.
Version Control	Git + GitHub	Set up on Day 1 of Sprint 1, not deferred to Sprint 3 — commit after every roadmap "Day" milestone below.
Containerization	Docker	Good hygiene for the reproducibility success metric, but treat as a Week 4+ polish item — don't let it consume Sprint 1-2 time.

6. Development Roadmap (Week-by-Week Execution Plan)

This is a 4-week sprint (with a bonus 4th and 5th week). Each day has a specific, actionable coding objective, a concrete deliverable, and a checkpoint to confirm before moving on. Commit to Git at the end of every day. Follow this strictly to avoid scope creep.

Environment setup (Day 0, before Sprint 1 begins): Install Python 3.10+, create a virtual environment (`python -m venv .venv`), initialize a Git repo and push an empty commit to GitHub, then install dependencies: `pip install numpy pandas scipy streamlit plotly matplotlib`.

Sprint 1: The Deterministic Physics Engine (Week 1)

formulas. Validate that fuel burn and tire degradation affect lap times correctly.

1. **Day 1 (File: tyre.py):** Create TyreCompound class with attributes: name, deg_coeff, cliff_threshold. Write a method time_penalty(age) that returns $a * age^2$. Plot this curve using Matplotlib to visually confirm exponential growth.
Deliverable: tyre.py + a saved PNG of the degradation curve. *Checkpoint:* curve should be flat early, then bend sharply upward near cliff_threshold.
2. **Day 2 (Files: track.py, car.py):** Define Track (base_lap_time, total_laps) and Car (fuel_capacity, fuel_burn_per_lap, weight_penalty_factor). Write the fuel correction formula: $lap_time += base * (fuel_remaining / total_weight) * 0.3$.
Deliverable: Two classes, each with a __repr__ for easy debugging. *Checkpoint:* instantiate a Monza Track and a default Car, print both.
3. **Day 3 (File: race_engine.py):** Write the simulate_deterministic(strategy, track, car) function. Loop through laps, decrement fuel, age tires, and when tire age exceeds stint length, add a **25-second pit penalty** and reset tire age.
Deliverable: Function returns a list of 70 individual lap times. *Checkpoint:* sum of lap times is a single sane float (roughly 90–100 minutes for a full race).
4. **Day 4 (File: main.py):** Hardcode two strategies: Strategy A = [Soft(15), Medium(25)] and Strategy B = [Medium(20), Hard(20)]. Run both simulations and print the total race time to the terminal.
Deliverable: Terminal output comparing both strategies' total times. *Checkpoint:* the two totals differ — if identical, a class attribute is likely being shared/mutated by reference.
5. **Day 5 (Validation):** Run the simulation manually with a calculator. If the fuel burns off, does the lap time decrease by roughly 0.05s per lap? If the tire cliff hits, does the lap time spike by 2 seconds? **If yes, Sprint 1 is complete.**
Deliverable: A short notes/sprint1_validation.md recording your manual spot-checks. *Checkpoint:* commit tagged sprint1-complete.

Sprint 2: The Stochastic Layer & Parallel Scaling (Weeks 2-3)

Goal: Introduce randomness and scale the simulation to run 100,000 times. Move away from Python loops

to NumPy vectorization for speed.

1. **Day 1 (Gaussian Noise):** Import `numpy.random`. Modify the lap time calculation: `lap_time += np.random.normal(0, 0.15)`. Run 1,000 iterations and store the total times in a list. Plot a histogram to see the normal distribution of race times.
Deliverable: Histogram PNG. *Checkpoint:* shape should visibly approach a bell curve as iteration count rises — this is your first live demonstration of the Law of Large Numbers.
2. **Day 2 (Safety Car - Poisson):** Implement the state machine. At the start of each lap, if `np.random.random() < 0.08`, trigger SC. For the next 3 laps, multiply tire degradation by `0` (no wear) and add 5 seconds to lap times. Test this by printing "SC DEPLOYED" to the console.
Deliverable: Working RACING → PIT_IN → PIT_STOP → PIT_OUT plus an SC_DEPLOYED state. *Checkpoint:* across 1,000 races, roughly 8% of laps should show an SC trigger — verify against the input probability.
3. **Day 3 (NumPy Vectorization):** Rewrite your lap-time loop to use NumPy arrays. Create an array of fuel levels for all 70 laps at once (`fuel_array = np.linspace(start, end, 70)`) and calculate all lap times in a single vectorized operation (no for loops).
Deliverable: `simulate_vectorized()` returning identical totals to the Day 3 (Sprint 1) loop version, on the same seed. *Checkpoint:* benchmark loop vs. vectorized on 1,000 iterations — vectorized should already be several times faster.
4. **Day 4 (Multiprocessing):** Wrap your simulation function. Use `from multiprocessing import Pool`. Split 100,000 simulations across your CPU cores (e.g., 8 cores running 12,500 each). Measure the runtime difference between sequential and parallel. Document the speedup.
Deliverable: A small timing table (sequential vs. parallel, in seconds) saved to `notes/speedup.md`.
Checkpoint: speedup roughly tracks core count, consistent with Amdahl's Law given the non-parallel setup overhead.
5. **Day 5 (Statistics & CI):** After running 100k races per strategy, calculate:
 1. Mean finishing time for Strategy A and B.
 2. Win Probability: `count(A_time < B_time) / 100000`.
 3. 95% Confidence Interval using `mean ± 1.96 * (std / sqrt(n))`.*Deliverable:* Print "Strategy A wins 67.3% ± 1.1%" to the terminal, results also written to a CSV.
Checkpoint: re-run twice — the win probability should be stable to within ~1%, confirming convergence. Commit tagged `sprint2-complete`.

Sprint 3: The Interactive Dashboard (Week 4)

Goal: Turn your terminal script into a professional, interactive web application that non-coders can use.

1. **Day 1 (Streamlit Setup):** Create `dashboard.py`. Install Streamlit. Add a title, a dropdown for "Track" (Monza vs Monaco), and a number input for "Race Length".
Deliverable: `streamlit run dashboard.py` opens a page with working inputs (no simulation logic wired up yet). *Checkpoint:* changing inputs updates the on-screen values instantly.
2. **Day 2 (Database Connection):** Instead of re-running simulations on every UI click (which is slow), run the simulations once and store the results in a **SQLite** database with columns: `sim_id`, `strategy_name`, `lap_number`, `lap_time`, `cumulative_time`. Index the `strategy_name` column.
Deliverable: `results.db` populated with 100k-run output from Sprint 2. *Checkpoint:* a query filtered on `strategy_name` returns in well under a second.
3. **Day 3 (Visualization - PDF Overlay):** Use Plotly to plot overlapping histograms. Set `histnorm='probability density'`. Overlay a `scipy.stats.gaussian_kde` smooth line on top of each histogram. This creates the "Probability Density Function" chart.
Deliverable: Overlapping PDF chart rendered live in the dashboard for Strategy A vs. B. *Checkpoint:* the two distributions should visibly separate if one strategy is clearly faster.
4. **Day 4 (Sensitivity Slider):** Add a slider for "Pit Stop Loss (seconds)" ranging from 20 to 30. When the user moves the slider, re-query the database and **recalculate** the win probability without re-running the physics (just adjust the final times). This proves your statistical agility.
Deliverable: Slider updates the win-probability display in under a second, no full re-simulation.
Checkpoint: moving the slider to the extremes (20s vs 30s) visibly shifts which strategy wins more often.
5. **Day 5 (Deployment & Demo):** Push your code to a public GitHub repo. Deploy the app to **Streamlit Cloud** (free). Record a 3-minute Loom/YouTube video walking through the app. Embed the video link in your README.
Deliverable: Live public URL + README with embedded video link. *Checkpoint:* open the deployed link in an incognito window to confirm it works with no local dependencies. Commit tagged `sprint3-complete`.

Sprint 4 (Bonus): Genetic Algorithm for Optimal Strategy (Week 5-6)

Goal: Let the computer find the mathematically perfect strategy instead of manually guessing. Build this

from scratch (not with a library like DEAP) so the Optimization Theory syllabus link stays concrete.

- Day 1 (Chromosome Encoding):** Define a strategy as a list of 3 integers: [Stint1_Laps, Stint2_Laps, Stint3_Laps]. Ensure they sum to 70. Generate a population of 100 random chromosomes.
Deliverable: generate_population(size=100). *Checkpoint:* every chromosome sums exactly to 70 laps with no negative stint lengths.
- Day 2 (Fitness Function):** Write a function that takes a chromosome, runs the Monte Carlo sim (1,000 times for speed), and returns the **average finishing time**. Lower time = higher fitness.
Deliverable: fitness(chromosome) -> float. *Checkpoint:* known-good strategies (e.g. Strategy A from Sprint 1) score sensibly relative to obviously bad ones (e.g. a single 70-lap stint on Softs).
- Day 3 (Selection & Crossover):** Implement "Tournament Selection" (pick 5 random, take the best). Implement "Crossover" by averaging two parent stint lengths and adding random Gaussian mutation.
Deliverable: select(), crossover(), mutate() functions, each unit-testable in isolation. *Checkpoint:* offspring chromosomes still sum to 70 after crossover and mutation (re-normalize if not).
- Day 4 (Evolution Loop):** Run the GA for 50 generations. Plot the "Best Fitness" and "Average Fitness" over generations. Watch the algorithm converge to a specific strategy.
Deliverable: Convergence plot PNG. *Checkpoint:* both curves should trend downward (lower finishing time = better) and flatten out by generation ~30-40.
- Day 5 (Final Comparison):** Take the AI-discovered strategy and pit it against the human strategies in a head-to-head 100k Monte Carlo run.
Deliverable: A final chart showing "Human Strategy vs AI Strategy" win probability, added to the dashboard as a third comparison option. *Checkpoint:* per the Section 7 success metric, the GA strategy should beat the human baseline by at least 0.5s average. Commit tagged sprint4-complete.

Sprint	Duration	Core Deliverable
Sprint 1 — Deterministic Engine	Week 1	Validated physics engine, zero randomness
Sprint 2 — Stochastic Layer & Scaling	Weeks 2-3	100k-iteration Monte Carlo engine with statistical output
Sprint 3 — Interactive Dashboard	Week 4	Deployed, public Streamlit app with live demo video
Sprint 4 — Genetic Algorithm (Bonus)	Weeks 5-6	AI-discovered strategy beating human baseline

7. Success Metrics & Portfolio Criteria

- ✔ **Functional:** App simulates a 70-lap race in under 200ms (pre-optimization).
- ✔ **Statistical:** Monte Carlo converges to stable win-probability within 1% margin after 50k iterations.
- ✔ **Visual:** Dashboard contains at least 3 chart types (PDF Histograms, Sensitivity Heatmaps, Lap-trace plots).
- ✔ **Documentation:** GitHub repo contains docs/ with LaTeX physics derivations and a video demo in README.
- ✔ **AI (Bonus):** Genetic Algorithm successfully finds a strategy that beats the human-chosen baseline by at least 0.5 seconds average.

8. Required Software & Initial Setup

Before writing a single line of code, ensure you have:

- **Python 3.10+** installed with pip.
- **VS Code** or PyCharm with the Python extension.

- A virtual environment created (`python -m venv .venv`).
- Dependencies installed via `pip install numpy pandas scipy streamlit plotly`.

Author: Jerome Maunga | **Project Status:** Ready for Sprint 1 Execution.

This document serves as the foundational blueprint for a BSc-level portfolio project, combining rigorous mathematical modeling with scalable software engineering.